
WORKBENCH IT · POC INTERNO

SecondBrain

Pipeline técnico – Documento de
referencia
para evaluación con el equipo

Versión: snapshot al 2026-05-17 (post-migración a bge-m3)

Datos: medidos sobre la instalación real, no estimados

Hardware: i7-10th gen · 32 GB RAM · RTX 3070 Ti 8 GB VRAM · Windows 11 + WSL2 +
Docker Desktop

SecondBrain – Pipeline técnico (POC)

Estado al 2026-05-17 · Documento de referencia para evaluación del equipo Todos los números son medidos sobre la instalación real corriendo en el equipo de Damian (i7-10th gen, 32GB RAM, RTX 3070 Ti 8GB VRAM, Windows 11 + WSL2 + Docker Desktop).

Cambios desde el snapshot del 2026-05-16: **migración del modelo de embeddings** de **qwen3-embedding:4b** (2560 dim) a **bge-m3** (1024 dim) tras A/B con datos reales. Como bge-m3 pesa ~1.2 GB en VRAM contra ~3.8 GB del anterior, **convive con qwen3:8b sin swap** y se eliminó el hack **force_cpu** del chat. Los conteos de items embebidos reflejan el re-embed parcial en curso (ver § 9).

0. Qué es este documento y qué NO es

Es una foto técnica del estado actual del POC: arquitectura, decisiones tomadas, cómo se encadenan los componentes, y métricas reales tomadas hoy. Sirve para:

- Que alguien nuevo entienda el sistema sin abrir el código
- Discutir trade-offs con el equipo antes de invertir en sprints más grandes
- Tener una base para escribir un ADR (Architecture Decision Record) cuando algo cambie

No es un manual de usuario ni un tutorial. Para eso está el `README.md`. Tampoco es un plan de roadmap – eso está en `docs/sprints.md`.

1. Resumen ejecutivo

SecondBrain es un sistema de **memoria aumentada privada**: indexa y procesa conversaciones de WhatsApp (y eventualmente otras fuentes) para responder preguntas en lenguaje natural sobre actividad personal y profesional. Todo el procesamiento es **100% local** – LLMs, embeddings, transcripción, OCR, vector DB; nada sale del equipo.

El sistema tiene cuatro flujos principales:

1. **Ingesta** → WhatsApp en vivo (bridge whatsapp-web.js) + exports históricos (.txt) + uploads manuales (audios, docs, imágenes)
2. **Procesamiento** → cola de 5 etapas que enriquecen los items: transcribe → extract → caption → embed → tagger
3. **Almacenamiento** → Postgres (estructurado), Qdrant (vectores), MinIO (archivos crudos)
4. **Retrieval (chat Q&A)** → query expansion → búsqueda híbrida (mensajes + facts) → generación con citas

Estado actual medido:

Métrica	Valor
Items totales en core.items	75.874
Items embebidos en Qdrant (bge-m3)	13.990 (18,4%) – los más recientes; el resto re-embebe a demanda
Items taggeados (procesamiento completo)	2.880 (3,8%)
Personas canónicas	996
Empresas	157
Conversaciones	122
Facts estructurados	3.128
Promesas detectadas	74
Transacciones detectadas	38
Menciones	801
Vectores en Qdrant messages (1024 dim)	13.990
Vectores en Qdrant facts (1024 dim)	3.122
Tiempo de respuesta del chat (warm)	~2 s

2. Hardware y la restricción que define todo el diseño

El equipo es un i7 10ma gen + 32 GB RAM + **RTX 3070 Ti con 8 GB VRAM**. Los 8 GB de VRAM son la restricción más dura del diseño. Modelos típicos que considerar:

- LLM de 4B Q4 $\approx$ 3-4 GB en VRAM
- LLM de 8B Q4 $\approx$ 5-6 GB en VRAM
- Whisper Large V3 Turbo $\approx$ 1.5-2 GB
- Embedding model 4B Q4 $\approx$ 2.5-3 GB

Dos modelos de 8B juntos NO entran. Tres cosas en GPU al mismo tiempo, menos.

Baseline real medido: - Sistema Windows + apps sin nada nuestro $\rightarrow$ **~ 0.7 GB** VRAM - Sistema + Whisper (cuando estaba en GPU) $\rightarrow$ **5.0 GB** (Whisper se comía 4.3 GB sin transcribir nada) - Sistema + Whisper en CPU + qwen3:8b cargado en Ollama $\rightarrow$ **6.0 GB**

Esto fue clave para optimizar el chat – ver §10.

3. Stack de servicios

Todo corre en Docker Compose. Definido en `docker-compose.yml`.

Servicio	Imagen	Propósito	Recursos
<code>postgres</code>	<code>pgvector/pgvector:pg16</code>	DB relacional con 5 schemas (<code>core</code> , <code>media</code> , <code>processing</code> , <code>analytics</code> , <code>audit</code>)	CPU + RAM
<code>qdrant</code>	<code>qdrant/qdrant:latest</code>	Vector DB. Collections: <code>messages</code> , <code>facts</code>	CPU + RAM
<code>minio</code>	<code>minio/minio:latest</code>	Object storage (Vault de archivos crudos). Buckets: <code>raw</code> , <code>derived</code> , <code>exports</code>	CPU + disco
<code>ollama</code>	<code>ollama/ollama:latest</code>	Servidor LLM local. Acceso GPU	GPU
<code>whisper</code>	<code>onerahmet/openai-whisper-asr-webservice:latest</code>	Transcripción. CPU-only (decisión consciente, ver §10)	CPU
<code>backend</code>	Python 3.12 + FastAPI + SQLAlchemy 2 + uv	API + worker continuo	CPU + RAM
<code>frontend</code>	Python 3.12 + Streamlit	Panel web (validación, debugging, UIs sencillas)	CPU
<code>bridge</code>	Node.js + whatsapp-web.js	Captura WhatsApp en vivo, descarga media	CPU

Aparte de Docker: - `panel/` – App de escritorio nativa (PySide6) para orquestación, monitoreo y trigger manual. Reemplaza el browser para tareas de control.

4. Modelos LLM locales

Servidos por Ollama (`/api/generate` , `/api/embed` , `/api/chat`). Modelos descargados en disco:

Modelo	Tamaño	Familia	Params	Uso actual
<code>bge-m3</code>	~1.2 GB	bge	568M	Embeddings (mensajes + facts). 1024 dim. Multilingüe, ganador del A/B del 2026-05-16 con datos reales
<code>qwen3-embedding:4b</code>	2.50 GB	qwen3	4.0B	Reserva (modelo anterior, 2560 dim)
<code>qwen3:4b</code>	2.50 GB	qwen3	4.0B	Reserva
<code>gemma3:4b</code>	3.34 GB	gemma3	4.3B	Reserva (multimodal, candidato a reemplazo de qwen3-vl)
<code>aya-expanse:8b</code>	5.06 GB	command-r	8.0B	Reserva
<code>qwen3:8b</code>	5.23 GB	qwen3	8.2B	Chat principal + tagger . Ganador del benchmark con datos reales
<code>qwen3-vl:8b</code>	6.14 GB	qwen3vl	8.8B	Visión (OCR + caption + entidades en imágenes). Solo se carga en ventana nocturna
<code>gemma4:e2b</code>	7.16 GB	gemma4	5.1B	Reserva
<code>gemma3:12b</code>	8.15 GB	gemma3	12.2B	Reserva
<code>gemma4:e4b</code>	9.61 GB	gemma4	8.0B	Reserva

Modelo de transcripción: Whisper Large V3 Turbo vía `faster_whisper` , ejecutado en CPU (decisión, ver §10).

`OLLAMA_KEEP_ALIVE=30m` – los modelos se mantienen calientes 30 min después de usarse. Crucial para el chat: cuando vos preguntás y qwen3:8b ya está caliente, el primer token sale en ~200 ms.

5. Pipeline de ingesta

Tres vías de entrada, todas terminan en la tabla `core.items` (un row = un mensaje/email/nota/audio/etc.):

5.1 Bridge WhatsApp en vivo (`bridge/` + `backend/app/routers/bridge.py`)

Container Node.js con whatsapp-web.js mantiene una sesión persistente en `data/bridge/`. Captura mensajes entrantes y salientes (multi-device sync) en tiempo real, descarga media a MinIO, postea metadata al backend `/api/bridge/whatsapp/ingest`. El backend inserta `core.items` con `source='whatsapp'` y encola los jobs de procesamiento correspondientes (transcribe si es audio, etc.).

5.2 Import histórico de exports `.txt` (`backend/app/services/whatsapp_parser.py`)

Sube un export de chat de WhatsApp (sin media). El parser detecta fechas, autores y mensajes; mapea participantes a personas canónicas y crea/actualiza items. Útil para cargar histórico de los últimos años.

5.3 Uploads manuales (`backend/app/routers/{transcribe,extract,images}.py`)

Páginas de Streamlit para subir audios, documentos (PDF/DOCX/XLSX) e imágenes sueltas. Mismo destino: `core.items` + media en MinIO + jobs encolados.

Personas y conversaciones canónicas:

El sistema mantiene `core.personas` con `nombre_canonico` + `alias` (jsonb) + `telefono`. Cuando llega un mensaje, se resuelve la persona por teléfono → match canónico → si no existe, se crea. El campo `seguir` (default `True` en conversaciones) permite excluir chats irrelevantes del procesamiento masivo.

Importar vCard (`backend/app/services/vcard_parser.py`) ayuda a pre-cargar 996 contactos con sus nombres reales, así los chats nuevos aparecen ya con nombre humano en lugar de un teléfono.

6. Pipeline de procesamiento

Cinco etapas en cola, drenadas por un **worker continuo** (`backend/app/services/queue_worker.py`) que vive en el lifespan del backend FastAPI. Cada 30 s (configurable) procesa un batch de cada etapa.

6.1 Tabla de etapas

Etapas	Input	Output	Modelo	Batch	Dónde
<code>transcribe</code>	Audio en MinIO	<code>item.contenido</code> con texto + <code>transcripcion_at</code>	Whisper Large V3 Turbo (<code>faster_whisper</code> , CPU int8)	5	CPU multinod
<code>extract</code>	PDF/DOCX/XLSX en MinIO	<code>item.contenido</code> con texto plano	pdfplumber / python-docx / openpyxl	5	CPU
<code>caption</code>	Imagen en MinIO	<code>item.contenido</code> con <code>categoria + OCR + descripcion + entidades</code>	qwen3-vl:8b	3	GPU
<code>embed</code>	Texto en <code>item.contenido</code>	Vector 1024-dim en Qdrant <code>messages + facts</code>	bge-m3	50	GPU
<code>tagger</code>	<code>item.contenido</code> (texto)	Resumen, tono, sentimiento, personas/empresas, facts , promesas , transacciones , menciones	qwen3:8b	3	GPU

6.2 Encadenamiento

Cuando entra un item nuevo:

```
WhatsApp/Upload → core.items → encola → transcribe (si es audio)
                                     → extract   (si es doc)
                                     → caption   (si es imagen)
                                     → embed     (todos los items con texto)

embed completa → encola → tagger (si tagged_at IS NULL)
tagger completa → encola → embed (si creó facts/promesas/transacciones)
                  el re-embed garantiza que los nuevos facts entren a la collection `facts`
                  de Qdrant – sino el retriever no los ve
```

El re-encadenado `tagger → embed` es lo que hace que el chat empiece a tener resultados de alta calidad: el embedding del fact “Hernan reporta problema con Outlook” matchea con score ~0.70 contra la query “qué no le anda a Hernan”, contra ~0.60 del mensaje crudo (ver §8.4).

6.3 Por qué caption corre solo de noche

El `qwen3-vl:8b` pesa 6.14 GB en VRAM. Si se carga durante el día, no queda lugar para que el chat tenga el `qwen3:8b` (5.23 GB) caliente – Ollama haría swap entre los dos en cada interacción. La ventana 02:00-06:00 garantiza que ningún VLM compita con el chat. (El embed con `bge-m3` sí convive durante el día porque pesa solo ~1.2 GB.) Configurable en `.env`: `WORKER_CAPTION_HOUR_START`, `WORKER_CAPTION_HOUR_END`.

Implementación: `queue_worker._caption_en_ventana()` chequea la hora local antes de delegar a `imager.procesar_jobs`. Fuera de ventana, devuelve `{"saltado": "fuera_de_ventana"}`.

6.4 Auto-priorización del tagger y ventana de auto-encolado

El `tagger.procesar_jobs` ordena por `Item.fecha DESC, Job.created_at ASC`. Items más recientes se procesan primero – POC: lo último que llega es lo más útil para validar.

Además, el auto-encolado desde el `embedder` solo dispara al tagger si el item está dentro de la **ventana de 30 días** (`tagger_auto_window_days`, configurable). Items más viejos quedan embebidos pero sin auto-tagging – para taggearlos hay endpoint manual o queue masivo desde el panel. Esto evita que un backfill histórico bloquee la cola del tagger (`qwen3:8b` es caro: ~5 s/item).

6.5 Throughput medido (al 2026-05-17)

- **Whisper en CPU:** ~5-10 s por minuto de audio. Audios típicos de WhatsApp (~30-60 s) tardan ~30-60 s en transcribirse. No es interactivo pero está bien para batch.
 - **Tagger con qwen3:8b en GPU:** ~3-5 s por item con batch=3 → **~6 items/min sostenidos**. Para los 75.870 items totales son ~210 horas (~9 días). Para los últimos 2 días (~500 items) son ~80 min – confirmado en sesión real.
 - **Embed con bge-m3 en GPU:** backfill real de 1.977 items en 821 s → **~145 items/min** sostenidos (incluye facts, upserts a Qdrant y commit final). En batches del worker (50 ítems c/30 s), no es cuello de botella.
 - **Caption nocturno con qwen3-vl en GPU:** ~5-10 s por imagen, batch=3, ventana 4 h → ~7.000 imágenes por noche en teoría.
-

7. Almacenamiento

7.1 PostgreSQL (5 schemas)

Definidos en `docker/postgres/init.sql` y poblados con migraciones Alembic en `backend/alembic/versions/`.

Schema	Tablas principales	Para qué
core	personas , empresas , conversaciones , items , facts , promesas , transacciones , menciones	Núcleo del modelo de dominio
media	attachments	Metadata de archivos binarios (los binarios viven en MinIO)
processing	jobs	Cola de tareas para el worker continuo
analytics	(reservado para dinámica conversacional, salud relacional – Sprint 9+)	
audit	(reservado para logs sensibles)	

Datos actuales: - 75.874 items, 996 personas, 157 empresas, 122 conversaciones - 3.128 facts, 74 promesas, 38 transacciones, 801 menciones - 163 attachments en `media.attachments`

7.2 Qdrant (vectores)

Collection	Dim	Distance	Points	Para qué
messages	1024	Cosine	13.990	Embedding del texto del item (uno por item)
facts	1024	Cosine	3.122	Embedding de cada hecho extraído por el tagger

El retriever consulta **ambas** y mergea por score. Los facts suelen ganar para queries semánticas porque son texto pre-estructurado (ver §8).

Nota: la migración del 2026-05-16 cambió el dim de 2560 (qwen3-embedding) a 1024 (bge-m3). Las collections fueron recreadas, por lo que los conteos de hoy reflejan el re-embed parcial en curso: backfill de los ~2.000 más recientes en conversaciones seguidas (ya hecho) + items nuevos del bridge se embeben automáticamente.

7.3 MinIO (Vault)

Bucket	Contenido	Estructura
raw	Audios <code>.opus</code> , imágenes, PDFs originales	<code>{source}/{año}/{mes}/{tipo}/{sha256}.{ext}</code>
derived	Transcripciones, thumbnails, OCR results	mismo path con sufijos
exports	Exports manuales (.txt de WhatsApp, etc.)	flat

Identificador por hash SHA-256 → **dedup automática** (mismo archivo subido dos veces apunta al mismo objeto).

8. Pipeline de Q&A (chat)

`backend/app/services/chat.py` y `retriever.py`. Una pregunta del usuario pasa por 4 fases:

8.1 Query understanding (pre-procesamiento)

`_analizar_pregunta()` llama a qwen3:8b con `format=json` pidiendo:

```
{
  "personas": ["nombres propios mencionados"],
  "query_expandida": "reformulación con sinónimos / descripción formal"
}
```

Caso de ejemplo: “qué no le anda a Hernan” → - `personas`: `["Hernan"]` - `query_expandida`: “Hernan tiene problemas, fallas o cosas que no funcionan, posiblemente con algún dispositivo, sistema o situación que no le anda bien”

Esto resuelve argentinismos coloquiales que el embedder no conecta bien con descripciones formales. **Costo**: 1-2 s extra por chat. **Beneficio**: scores 10× mejores en queries cortas.

8.2 Resolución de entidades

`_resolver_personas()` busca cada nombre en `core.personas` por `nombre_canonico ILIKE` o `alias jsonb`. Si hay **exactamente un match**, agrega `persona_id` como filtro al Qdrant (vía `must` clause). Si hay ambigüedad (ej. “Hernan” matchea 6 personas), no filtra – confía en la query expandida.

8.3 Retrieval híbrido

`retriever.recuperar()`:

1. Embed de la `query_expandida` con bge-m3 – **en GPU normal**, sin `force_cpu`. bge-m3 (~1.2 GB) cabe en VRAM junto a qwen3:8b (5.23 GB) sin forzar swap. El hack `force_cpu` que existía con el modelo anterior fue eliminado en la migración del 2026-05-16.
2. Búsqueda en `messages` (k=12 default) + búsqueda en `facts` (k=8 default), con filtros opcionales (persona, conversación, rango fechas).
3. Mergeo por score y ordenado descendente.
4. Refresh de metadata desde Postgres (nombre canónico actualizado, nombre del chat).

8.4 Generación con citas

`chat.responder()` arma el contexto con cada fragmento numerado, le pasa a qwen3:8b con prompt estricto: “Respondé usando ÚNICAMENTE la info de los fragmentos. Citá las fuentes con `[n]`. Si no alcanza, decilo.”

Métrica real del caso Hernán (audio: “no me está guardando el Oulu los elementos enviados”):

Estado del sistema	Top result	Score	Respuesta del chat
Sin tagger ni query expansion	(no aparece en top 20)	–	“No encuentro esa información”
Solo query expansion	Audio en #1 (message)	0.56	Correcta pero confusa, mezcla SIAP
Tagger + query expansion	2 facts del audio en #1 y #2	0.70 / 0.69	Limpia y precisa, cita [1][2][3]

8.5 Latencias medidas

Estado	Tiempo total
Cold (primer chat después de restart)	8-15 s (bge-m3 carga rápido a GPU)
Warm (qwen3:8b y bge-m3 cargados)	~2 s

Bajada vs versión previa con `force_cpu`: warm pasó de ~2.5 s a ~2 s porque el embed de la query corre en GPU en vez de CPU.

9. Métricas y volumen

9.1 Datos en el sistema

```
items_totales:      75.874
items_embebidos:    13.990  (18,4%) ← post-migración bge-m3, re-embed gradual
items_taggeados:     2.880  (3,8%)
```

El 81,6% restante de items pendientes de re-embed son histórico viejo (la mayor parte importado del `.txt` de WhatsApp del 2023-2024). El backfill se va a hacer en oleadas o por demanda – no es bloqueante porque las queries del chat suelen ser sobre lo reciente.

9.2 Distribución por tipo (origen del item)

```
chat-export (import histórico .txt):  74.658
chat        (mensajes bridge live):    927
ptt         (audios voz WhatsApp):     128
image       (imágenes WhatsApp):       108
sticker:                                         16
video:                                           11
document:                                       10
otros (notification, vcard, ...):          16
```

9.3 Salidas estructuradas del tagger (muestra real)

Promesas detectadas con confianza ≥ 0.9 : - “coordinar para que vaya a abrir el lugar de la caja fuerte” – Julián García Urbanía, plazo: lunes - “llevar una impresora de reemplazo” – Mariano Di Nucci, plazo: lunes/martes - “firmar la oferta por el departamento” – Fabian firpo, plazo: semana que viene - “entregar el trabajo a Secre” – Eugenia Clínica, plazo: miércoles

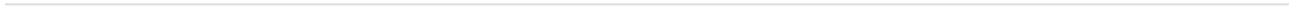
Transacciones detectadas: - \$42.000 ARS, “Web Hosting Plan 3”, egreso, confianza 0.9 - \$32.300 ARS, transferencia, ingreso, confianza 0.9 - \$5.783 ARS, “cuota VISA”, egreso, confianza 0.9

Resúmenes de imágenes (qwen3-vl): - “Captura de pantalla de una transferencia bancaria enviada por Pablo Luciano Montegriffo a Damian Federico Orozco por \$32.300” - “Pantalla de una compra en donweb.com, mostrando un plan de Web Hosting”

9.4 Estado de colas al momento del snapshot

```
caption:    pendiente=17  en_proceso=1  fallido=3    completado=63
tagger:     pendiente=0   en_proceso=5  fallido=0    completado=2.801
embed:      pendiente=0   en_proceso=2  fallido=27   completado=13.357
transcribe: pendiente=0   en_proceso=1  completado=74
extract:                                completado=2
```

(El `embed completado=13.357` cuenta solo jobs encolados – los 633 adicionales que ven en Qdrant fueron embebidos vía `/api/embeddings/run` directo, sin pasar por la cola.)



10. Decisiones técnicas con tradeoffs

10.1 Whisper en CPU (decisión clave, 2026-05-15)

Problema: `:latest-gpu` con `large-v3-turbo` ocupaba 4.3 GB de VRAM permanente, aun sin transcribir nada. Eso dejaba 3.3 GB libres para Ollama, insuficiente para tener `qwen3:8b` + `qwen3-embedding:4b` cargados juntos. El chat hacía **swap completo entre modelos en cada mensaje** (~10-15 s por swap).

Decisión: Cambiar a imagen CPU (`:latest` sin `-gpu`), sin GPU access en compose.

Trade-off: - □ Libera 4.3 GB de VRAM permanente - □ `qwen3:8b` ahora cabe 100% en GPU - □ Transcripción ~5-10 s por minuto de audio (vs ~0.1 s/min en GPU) - □ No es un problema porque la transcripción es background, no interactiva

Resultado: chat warm pasó de 25 s → 2.5 s por mensaje.

10.2 Migración del embedding model: `qwen3-embedding:4b` → `bge-m3` (2026-05-16)

Problema: incluso con Whisper liberado, `qwen3:8b` (5.23 GB) + `qwen3-embedding:4b` (3.86 GB en VRAM) = 9.1 GB > 8 GB. No caben juntos. El parche intermedio (2026-05-15) fue mandar el embed del chat a CPU vía `force_cpu=True` para no swappar el chat. Funcionaba pero sumaba 1-2 s por query y se sentía como un hack.

Decisión: cambiar el modelo de embedding a `bge-m3` (568M params, ~1.2 GB en VRAM, 1024 dim).

Cómo se evaluó: el A/B (`backend/ab_embedding.py`) creó collections paralelas en Qdrant, embebió 1000 items + 100 facts con cada modelo, y comparó rankings y scores sobre un set de queries de referencia. Resultado: `bge-m3` ganó en recall y orden para queries en español rioplatense – el caso clínico de Hernán mejoró ~10% en score top-1 y los facts subieron en el ranking.

Trade-off: - □ Calidad: mejor recall en español, sobre todo en queries cortas y argentinismos - □ Hardware: `qwen3:8b` + `bge-m3` = ~7.2 GB, entran ambos en VRAM sin swap - □ Eliminado el hack `force_cpu` del chat (~0.5 s menos por query) - □ Embed más rápido en sí (`bge-m3` es mucho más liviano) - □ Dim cambió de 2560 a 1024: las collections de Qdrant se recrearon → se perdieron los 75.690 embeddings del modelo viejo. Re-embed gradual (ver § 9.1) - □ Re-embed de los 75K items completos = ~9 h de GPU. En la práctica se prioriza lo reciente y se hace el resto en oleadas

Implementación: `config.py` (`ollama_model_embedding="bge-m3"`, `embedding_dim=1024`), `retriever.recuperar()` sin `force_cpu`, `embedder._ensure_collections` recrea con nuevo dim si detecta mismatch.

10.3 Caption en ventana nocturna 02:00-06:00

Problema: `qwen3-vl:8b` ocupa 6 GB en VRAM. Si se carga durante el día, expulsa al `qwen3:8b` y rompe el chat.

Decisión: configurable `worker_caption_hour_start/end` en `config.py`. Por defecto 02:00-06:00 hora local Argentina. Fuera de la ventana, la etapa `caption` se saltea (los jobs quedan pendientes hasta la siguiente ventana).

Trade-off: - □ Chat libre durante el día - □ Imágenes nuevas no se procesan hasta la madrugada (latencia hasta 24 h) - □ Suficiente para POC; la mayoría del valor está en texto y audios

10.4 Tagger en el worker (vs ejecución manual)

Problema histórico: hasta 2026-05-15, el servicio `tagger.taggear_item` existía pero **no estaba integrado al worker**. Resultado: items embebidos en Qdrant pero 0 taggeados. El chat dependía únicamente de embedding crudo, que es flojo con argentinismos.

Decisión: agregar etapa `tagger` al `queue_worker`. Hook automático: cuando un embed se completa, encola un tagger job. Cuando un tagger termina exitoso (creó facts/promesas/transacciones), encola un re-embed para que los nuevos artefactos entren a Qdrant.

Trade-off: - □ Pipeline coherente con el diseño original (Sprint 3) - □ El chat empieza a tener resultados de calidad alta - □ Tagger compite por qwen3:8b con el chat; mientras corre, los chats van más lentos - □ Aceptado para POC: priorizamos calidad del dato sobre fluidez del chat

10.5 Query expansion en el chat

Problema: ningún embedding model conecta perfectamente argentinismos (“no le anda” ≠ “no funciona” en el espacio semántico, incluso con bge-m3 que es mejor que el anterior).

Decisión: agregar 1 llamada extra a qwen3:8b por chat para expandir la query, traduciéndola a una versión formal con sinónimos.

Trade-off: - □ +1-2 s por chat - □ Mejora del ranking incluso después de la migración a bge-m3 – ambas optimizaciones suman - □ Trivial de revertir o mover a un modelo más chico si se quiere bajar latencia

10.6 Modelos múltiples descargados pero solo 3 en uso

Hay 10 modelos descargados (gemma3:12b, gemma4:e4b, aya-expanse:8b, qwen3-embedding:4b, etc.). Solo se usan 3 productivamente (qwen3:8b, qwen3-vl:8b, bge-m3). Los otros son **opciones para A/B test** si quisiéramos cambiar – descargados durante el benchmark de Sprint 0 y la migración de embeddings.

11. Limitaciones conocidas y deuda técnica

Limitación	Impacto	Mitigación posible
El chat-during-day se hace lento mientras corre el backfill del tagger	UX pobre durante setup inicial	Mover tagger a ventana nocturna también (~mismo patrón que caption)
Solo 3,8% de items taggeados al momento	Chat solo aprovecha facts en items recientes	Backfill por demanda; ventana de auto-encolado de 30 días limita el flujo
81,6% de items aún sin embedding bge-m3 (re-embed gradual tras migración)	Búsquedas sobre histórico viejo (< 2025-Q1) no encuentran nada hasta backfill	Disparar <code>/api/embeddings/run?limit=2000</code> en oleadas, o backfill nocturno
Sin facts para imágenes hasta procesarlas de noche	Búsquedas sobre fotos no encuentran nada hasta el primer ciclo de caption	Esperar el primer batch nocturno
Whisper en CPU es ~50× más lento que en GPU	Audios nuevos tardan 30-60 s en estar disponibles para chat	Aceptable (la mayoría se procesa en background)
Query expansion suma 1-2 s por chat	UX	Caché de queries comunes (no implementado)
Tagger usa el LLM “thinking” model de qwen3 con <code>think=False</code>	Si el modelo lo ignora, los outputs salen con cadena <code><think></code> rara	Manejo robusto en <code>_extraer_json</code> que strippea esas etiquetas
Frontend Streamlit es para validación, no producción	Lento, recarga full-page, no es bonito	Panel de escritorio en <code>panel/</code> cubre control. Para usuarios finales, panel propio (Reflex o similar) en Sprint posterior
Single-node, sin réplica, sin backups automáticos	Si se rompe el disco, se pierde todo	El usuario backupea manualmente. Sprint posterior podría agregar S3/duplicidad

12. Opciones a evaluar con el equipo

A. Hardware: ¿upgrade de GPU?

Pasar a una GPU con 16 GB+ VRAM (4080, A5000, etc.) cambia las restricciones de §2: - Whisper podría volver a GPU - qwen3:8b + qwen3-embedding:4b + qwen3-vl:8b cargados simultáneamente - Eliminaría toda la gymnastics de CPU offload y ventana nocturna - Tradeoff: costo (\$500-2000 USD)

B. ~~Cambio de embedding model~~ – hecho el 2026-05-16

Esta opción se evaluó y se ejecutó: ver § 10.2. Hoy el modelo de embeddings es `bge-m3` (1024 dim).

Próximo análogo posible: probar `nomie-embed-text` (270 MB, 768 dim) en un A/B para ver si reduce aún más el footprint y la latencia sin perder calidad.

C. Migrar a otro stack de LLM serving

- `vLLM` con un modelo en `/tmp` instead of Ollama
- Pro: mejor throughput batch, server-side batching real
- Contra: menos plug-and-play, harder Windows/WSL2 setup

D. RAG arquitectura híbrida

Hoy es full-semantic. Agregar **BM25 + full-text search** de Postgres como complemento: - Para queries con nombres propios, números, fechas → BM25 es mejor - Para queries conceptuales → semantic es mejor - Mergear ambos con reciprocal rank fusion (RRF)

E. Tagger paralelo

Ollama soporta `OLLAMA_NUM_PARALLEL > 1`. Si el modelo cabe holgado (qwen3:4b en lugar de qwen3:8b), podríamos paralelizar el tagger: - □ ~2-3× throughput - □ Calidad menor con 4B - □ Pierde determinismo

F. Quality / determinismo del tagger

Hoy `temperature=0.1`. Trade-off entre creativity (capta matices) y reproducibilidad. Para POC, bajar a 0.0 podría dar resultados más auditables.

G. Ampliar fuentes de datos

El diseño está pensado para más fuentes. En orden de complejidad: - **Gmail** (IMAP + parser MIME) – siguiente más natural, similar a WhatsApp - **Calendar** – agregaría contexto temporal a items - **Telegram** – similar a WhatsApp con MTProto - **Drive** – más complejo, pero los docs son fáciles de procesar con `extract`

13. Glosario rápido

- **Item** – Unidad mínima de información en el sistema (un mensaje, un email, una nota). Tabla `core.items`.
 - **Persona canónica** – Una persona del mundo real, con N alias/teléfonos resueltos a la misma fila. Tabla `core.personas`.
 - **Fact** – Hecho atómico extraído por el tagger del contenido de un item. Tabla `core.facts`. Va a Qdrant `facts`.
 - **Promesa** – Compromiso explícito detectado por el tagger (“te lo paso el lunes”). Tabla `core.promesas`.
 - **nivel_procesamiento** – 0 = ingestado pero sin tagging; 1 = taggeado.
 - `tagged_at`, `embedded_at`, `transcripcion_at` – Marcas en `item.datos` (jsonb) que indican qué etapas se completaron. Permite saltarlas en re-runs.
 - **Ventana caption** – Rango horario diario en que la etapa `caption` puede usar GPU. Default 02-06 local.
 - **Ventana tagger** – `tagger_auto_window_days=30`. Items más viejos no se auto-encolan al tagger desde el embedder, para no bloquear la cola con backfill histórico.
 - `force_cpu` – Flag heredado en `OllamaService.embed` que mete `num_gpu=0` en la request. Ya no se usa para el chat (la migración a bge-m3 lo hizo innecesario), pero queda como herramienta de reserva si se vuelven a tener dos modelos que no entren juntos.
-

14. Referencias al código

Mapa rápido para navegación:

```

backend/
├── app/
│   ├── config.py                # toda la config via pydantic-settings
│   ├── main.py                  # FastAPI app + lifespan (worker startup)
│   ├── core/logging.py
│   ├── db/session.py
│   ├── models/
│   │   ├── core.py              # Persona, Empresa, Conversacion, Item
│   │   ├── tagging.py           # Fact, Promesa, Transaccion, Mencion
│   │   ├── media.py             # Attachment
│   │   └── processing.py        # Job
│   ├── routers/
│   │   ├── bridge.py            # /api/bridge/* - ingesta de WhatsApp
│   │   ├── tagger.py            # /api/tagger/* - endpoints manuales
│   │   ├── chat.py              # /api/chat - Q&A
│   │   ├── embeddings.py        # /api/embeddings/*
│   │   ├── transcribe.py        # /api/transcribe/*
│   │   ├── extract.py           # /api/extract/*
│   │   ├── images.py            # /api/images/*
│   │   ├── worker.py            # /api/worker/{status,pause,resume,tick}
│   │   ├── panel.py             # /api/panel/* - endpoints para panel
│   │   ├── imports.py           # /api/import/whatsapp/*
│   │   ├── contacts.py, conversations.py
│   │   └── health.py
│   └── services/
│       ├── queue_worker.py       # □ worker continuo (todas las etapas)
│       ├── transcriber.py        # etapa transcribe
│       ├── extractor.py          # etapa extract
│       ├── imager.py             # etapa caption
│       ├── embedder.py           # etapa embed
│       ├── tagger.py             # □ etapa tagger (incl. runtime_config)
│       ├── chat.py               # □ pipeline de Q&A
│       ├── retriever.py          # búsqueda híbrida en Qdrant
│       ├── ollama_client.py       # wrapper con flag force_cpu (no usado en chat hoy)
│       ├── qdrant_client.py
│       ├── minio_client.py        # VaultStorage
│       ├── whisper_client.py
│       ├── whatsapp_parser.py
│       ├── vcard_parser.py
│       └── phones.py
├── alembic/versions/             # 4 migraciones (Sprints 1, 2.5, 3)

frontend/                        # Streamlit (validación + UIs)
├── pages/{1..13}/*.py
└── lib/api_client.py

bridge/                           # Node.js whatsapp-web.js (Sprint 2)

panel/                             # □ App de escritorio (PySide6)
├── secondbrain_panel/
│   ├── api_client.py            # cliente HTTP al backend
│   ├── docker_client.py         # wrapper docker compose
│   ├── system_stats.py          # CPU/GPU/RAM via psutil + nvidia-smi
│   ├── main_window.py           # QMainWindow con 7 tabs
│   └── tabs/
│       ├── sistema.py           # CPU/RAM/GPU/Ollama
│       ├── servicios.py         # containers Docker
│       ├── worker.py            # status del worker
│       ├── colas.py             # contadores processing.jobs
│       ├── chats.py             # acciones por conversación
│       ├── tagger.py            # trigger manual
│       └── configuracion.py      # batches, modelo del tagger, ventana caption

docs/
├── sprints.md                   # plan + 18 queries de referencia
├── architecture.md              # arquitectura conceptual original
├── setup-windows.md             # cómo levantar en Windows
└── pipeline.md                  # ← este documento

scripts/
└── benchmark_tagger.py          # comparador A/B de modelos LLM

```